Causal Metadata Overhead Under Churn in Distributed Key-Value Stores

Eric Nelson
Department of Computer Science
The University of Texas at Austin
Austin, Texas
enelson@utexas.edu

Pallavi Desai
Department of Computer Science
The University of Texas at Austin
Austin, Texas
pallavi.desai@utexas.edu

***Abstract*—Version vectors are a common mechanism for tracking causality in highly available distributed key-value stores, but their metadata overhead grows under membership churn, creating a trade-off between causal history and scalability. Dotted Version Vectors (DVVs) address this problem by more efficiently representing concurrent updates, while lease-based pruning attempts to further control metadata growth by expiring old causal history.**

This project presents a discrete-event simulator for evaluating version vector and DVV-based causal clocks under dynamic churn environments. We quantify the metadata advantage of DVVs for sibling representation and evaluate lease-pruned DVV variants across different churn profiles and timeout configurations. Our results show that DVVs preserve the correctness of exact version vectors while substantially reducing sibling metadata overhead, whereas lease-based pruning can effectively bound metadata growth under churn at the cost of progressively losing causal history. We additionally evaluate an adaptive lease mechanism and find that it fails to meaningfully improve this trade-off (see Appendix).
Code:https://github.com/pallavidesai-ut/distributed-computing-project/

I. Introduction

Modern distributed key-value stores prioritize availability and partition tolerance by allowing multiple replicas to accept concurrent updates independently. To reconcile these updates correctly, systems must track causal relationships between writes. Logical clocks such as Version Vectors (VVs) capture causality exactly, but their metadata scales with the number of participating actors: per-client schemes grow rapidly with active clients, while per-replica schemes cannot distinguish concurrent writes coordinated by the same replica. In long-running systems with dynamic membership, where clients and replicas continuously join and leave, both modes grow linearly with membership and fail to scale. Dotted Version Vectors (DVVs) were introduced to address this by pairing a unique per-write identifier with a compact causal summary, allowing concurrent writes through the same replica to be distinguished without inflating metadata across siblings. DVVs preserve exact causality for replicated stores, but their summaries still accumulate entries over time under sustained churn unless older causal history is actively pruned. Production systems such as Riak pair DVVs with ad-hoc pruning heuristics like time-to-live expiration and background anti-entropy, but the trade-off between metadata growth and causal history loss under such pruning has not been systematically evaluated in the literature. This paper addresses that gap. We ask two questions: how large is the metadata advantage of DVVs over traditional version vectors when representing concurrent siblings, and how does lease-based pruning trade ancestry recall for bounded metadata growth as membership dynamics vary? We answer both through a discrete-event simulation of a partially replicated key-value store under a range of churn profiles.

II. Motivation and Related Work

To motivate these questions, we review prior approaches to causal tracking, metadata compaction, and scaling trade-offs with clock variants.

A. Foundational Causality and the Scaling Bottleneck

The tracking of causality in distributed systems originated with Lamport's scalar clocks, which provided a total ordering of events but lacked the ability to identify concurrent execution [1]. This limitation was resolved by the introduction of Vector Clocks (VCs) by Fidge and Mattern, which accurately capture concurrent relationship by maintaining an array of logical counters, one for each participating process [2], [3]. However, in environments with membership churn, the metadata overhead of VCs grows linearly, with $O(N)$ entries where N represents the lifetime sum of all unique processes/nodes. This results in unbounded metadata growth as nodes join and leave, rendering pure VCs unsuitable for long-running, dynamic real world systems where membership churn is common.

B. Logical Clocks for Dynamic Membership

To address the $O(N)$ scaling constraint, several mechanisms have been proposed to adapt vector-like structures to dynamic environments. Landes introduced Dynamic Vector Clocks, formalizing the addition and retirement of entries using distributed consensus, though the coordination overhead proved prohibitive for highly partitioned systems [4]. Torres-Rojas and Ahamad proposed Plausible Clocks, which guarantee a constant-size metadata overhead by mapping an unbounded number of processes to a fixed-size vector [5]. While highly space-efficient, Plausible Clocks trade consistency for size,

leading to false positives in conflict detection when distinct processes map to the same vector index.

To address causal tracking under heavy churn, Almeida *et al.* introduced Interval Tree Clocks (ITC) [6]. ITC eliminates the need for global identifiers entirely, allowing nodes to dynamically "fork" and "join" identity spaces by splitting ownership into a tree-like structure. While ITC can keep metadata compact under balanced operation, asymmetric churn patterns may cause the identifier structure to become increasingly fragmented, leading to larger bit-string representations over time.

C. Version Vectors and Optimistic Replication

In the context of distributed data stores, vector clocks are typically adapted into Version Vectors (VVs) to manage optimistic replication and conflict resolution, as originally implemented in the LOCUS distributed operating system [7]. Modern distributed architectures, heavily influenced by Dynamo [8], initially relied on traditional VVs but quickly encountered severe metadata bloat and "sibling explosion" when managing transient clients or ephemeral storage nodes (both high churn environments).

Preguiça et al. addressed this for highly available key-value stores by introducing Dotted Version Vectors (DVVs) [9]. A DVV pairs a *dot*, a unique event identifier assigned by the coordinating replica, with a causal context summarizing the history that event observed, allowing concurrent writes through the same replica to be uniquely identified without a full version vector. Metadata size (for a given replica) is thereby bounded by the number of coordinating replicas rather than the number of active clients, an approach adopted in Dynamo-style stores such as Riak. However, while compact for representing concurrent relationships per node, DVV still scales linearly with unbounded metadata as the summary vector gradually grows under membership churn without any garbage collection or pruning mechanism.

D. Metadata Pruning and Garbage Collection Strategies

While structural innovations like DVVs compress causal history, systems experiencing continuous, long-term churn must eventually rely on active pruning to prevent memory exhaustion and network saturation from bloated metadata. In practice, industry systems rely on heuristics rather than formally verified protocols. For instance, Apache Cassandra utilizes a passive time-to-live (TTL) expiration mechanism to aggressively delete tombstones and causal metadata [10]. Similarly, systems like Riak pair DVVs with active anti-entropy background services using Merkle Trees to periodically reconcile state [11].

These prior implementations highlight a gap in the academic literature we aim to study: the tradeoff between causal safety and active pruning. While extensive research exists on ensuring causal correctness under the assumption of persistent metadata, there is limited evaluation of expiration-driven pruning mechanisms. Current literature treats metadata expiration either as a static, blunt-force TTL or relies on expensive fallback state transfers. Our work seeks to evaluate the tradeoffs of causal history preservation and metadata growth under different churn environments for standard DVV heuristic pruning methods.

III. Definitions

1) Ancestry Recall: Recall measures the fraction of true causal ancestors that are correctly encoded in a version's clock metadata. Formally, for a version v with true causal history $H(v)$ and clock-encoded history $\hat{H}(v)$:

$$\text{Recall}(v) = \frac{|H(v) \cap \hat{H}(v)|}{|H(v)|} \tag{1}$$

A recall of 1.0 means the clock perfectly encodes the full causal ancestry of v. A recall below 1.0 means some true ancestors have been forgotten, either through pruning (Lease-DVV) or through coarse actor identity (VV). Forgotten ancestors cause the store to retain stale siblings that should have been superseded, and in the worst case cause lost updates when a later write fails to recognize an earlier one as its ancestor.

2) Stale-Sibling Rate: The stale-sibling rate is the fraction of stored versions that are true causal ancestors of another stored version for the same key, i.e., versions that should have been garbage-collected but were not because the clock forgot the ancestry link. High stale-sibling rates indicate compounding reconciliation debt: siblings accumulate faster than merge writes can resolve them.

3) Metadata Size: Metadata size is measured as the serialized JSON byte length of the clock payload, excluding the clock-type label. This is a relative proxy for metadata overhead. We report both mean bytes per write and 95th-percentile bytes per write.

4) Actor Entries: An actor entry is one component of a version vector or DVV context, identified by a unique actor ID (frame of reference for each experiment). For client-actor VV, actors are client/session identifiers (Experiment 1). For server-actor representations (Experiment 2: VV, DVV, Lease-DVV), actors are replica node identifiers. The number of actor entries per clock is the primary driver of metadata size growth.

IV. Experimental Design

We run two main experiments. One to prove that DVV is more efficient in representing concurrent client writes (siblings), and another to show the causal history tradeoff vs metadata-size of from lease-based pruning.

A. Experiment 1: Client-Actor VV vs DVV Sibling-Set Metadata

The first experiment demonstrates DVV clear metadata advantage over VV: when many concurrent siblings sharing the same causal context. In this setup the a "Client-Actor" VV must store a version vector for each client accessing it's data as well as each replica to de-conflict concurrent writes. DVV natively handles it by comparing only the conflicting "dots".

We sweep the number of ancestor actors over $\{1, 4, 16, 64, 128\}$ and the number of concurrent siblings over $\{1, 2, 4, 8, 16, 32, 64\}$. For each grid point, we compare two encodings. The client-actor VV encoding stores a full version vector with every sibling. The DVV encoding stores the shared causal context once and one dot per sibling. We report total sibling-set metadata bytes, metadata component counts, and percent metadata reduction of shared DVV relative to repeated VV. We also verify that the shared DVV encoding reconstructs the original per-version contexts exactly, so this experiment measures metadata cost for total historical preservation of DVV vs VV (which requires O(n * m) for n clients and m replicas space for Client-Actor VV as we will see).

B. Experiment 2: VV, DVV, and Lease-DVV Under Churn

The second experiment asks how full causal tracking behaves when replica membership changes over time, and what is lost when we add pruning. We run the same key-value-store workload under stable, low, sustained, and burst churn, comparing VV, DVV, and fixed Lease-DVV. In this experiment, VV is a physical replica-actor version vector that keeps an actor-to-counter entry for each replica actor in an object's history. This VV variant is causally weaker than DVV because multiple independent client writes coordinated by the same replica share the same actor counter; a later counter value can make concurrent writes look causally ordered. DVV fixes this by representing each write with an explicit dot in addition to the causal context. Lease-DVV starts from the DVV representation but expires old actor entries after a lease window. This makes Lease-DVV weaker in a different way: it can reduce metadata, but it may forget ancestry that DVV would retain.

We introduce Lease-DVV as a tunable approximation that trades recall for metadata reduction. This stage runs three lease configurations ($L \in \{8, 16, 32\}$) across all four churn profiles, producing a lease-duration $\times$ churn-profile result grid. The central question is whether a moderate lease can recover most of DVV's metadata savings while keeping recall loss within an acceptable bound. Results are presented as paired metadata and recall curves across lease durations (to show the tradeoff shape) and as a summary table of mean recall by lease and churn profile (to show the interaction with membership dynamics).

C. Experiment Simulation Configuration

The simulator is a discrete-event model of a partially replicated key-value store. All experiments share the following base configuration unless otherwise noted.

1) Churn Profiles: Each experiment is run under four membership churn profiles that vary the rate at which replicas join and leave the cluster. The burst profile is the adversarial case for Lease-DVV because it combines periodic mass departures with subsequent rejoins under the same actor ID, which is the exact scenario that causes pruned history to conflict with returning state.

TABLE I
BASE SIMULATOR CONFIGURATION.

Parameter	Value
Replication factor	4
Client/session pool	128
Hot-key probability	0.3
Burst writers	8 concurrent clients
Merge probability	0.4
Network delay	Uniform $[1, 5]$ sim-time units
Random seeds	3 per configuration

TABLE II
CHURN PROFILES (SEE APPENDIX FOR DETAILS)

Profile	Description
Stable	No joins or leaves. Replica membership is fixed for the duration of the run.
Low	Slow continuous joins and leaves. Membership changes infrequently.
Sustained	Higher continuous churn. Replicas join and leave at a rate that keeps a fraction of the cluster in flux throughout the run.
Burst	Periodic removal of multiple replicas followed by rejoins, plus low background churn between bursts. A fraction of departed nodes rejoin with the same actor ID.

V. IMPLEMENTATION

We implemented a discrete-event simulator for a partially replicated key-value store. The simulator models replicas, client read-then-write operations, asynchronous replication, membership churn, hot-key contention, and merge writes. Each write reads the current sibling set for a key, combines it with the client's session context, issues a new clock stamp, installs the new version locally, and replicates it to a configurable number of active replicas after a random network delay.

The simulator separates ground truth from clock metadata. Each version records its true causal history, but replicas use only clock stamps for conflict resolution. Incoming versions replace dominated siblings, are dropped when dominated, and are retained otherwise as concurrent siblings. Comparing these clock-based decisions against the hidden ground truth lets us measure ancestry recall, missed conflicts, stale siblings, and metadata cost.

All clock variants use the same interface: build a read context, issue a new stamp, compare two stamps, and optionally prune metadata. VV stores a sparse actor-to-counter map. DVV stores a causal summary plus an explicit dot for the new write. Lease-DVV extends DVV by removing actor entries whose leases have expired before issuing a new stamp. This bounds metadata under churn but can forget true ancestry. Adaptive Lease-DVV uses the same pruning mechanism, but adjusts the effective lease using actor silence, sibling pressure, observed latency, churn, and metadata width.

Each run emits CSV traces for writes, deliveries, version-resolution decisions, membership events, snapshots, and ancestry accuracy. A separate analysis pass computes the aggregate metadata and causal-accuracy metrics reported in the evalua-

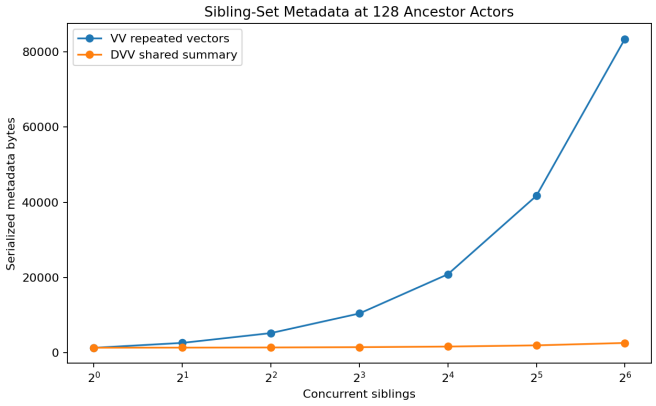

Fig. 1. Experiment 1: Total sibling-set metadata at 128 ancestor actors. VV repeats the full context in each sibling scaling quadratically; DVV shares one summary context across all siblings with one dot per sibling.

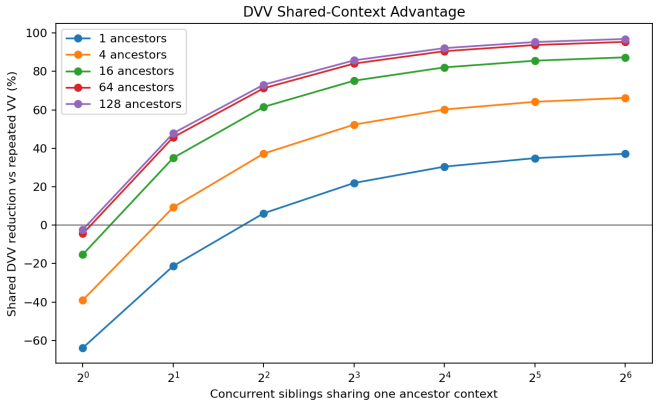

Fig. 2. Experiment 1: DVV metadata reduction by 95% relative to repeated VV, as a function of concurrent sibling count and ancestor actor count. Negative values indicate DVV costs more than VV in that region.

tion. **Not all of these features/analysis were used for this report.**

VI. Evaluation

A. Experiment 1: Client-Actor VV vs DVV Sibling-Set Metadata

Experiment 1 evaluates the metadata cost of representing large sibling sets under high concurrency. Figure 1 compares the total serialized metadata required to store a sibling set when using repeated client-actor VVs versus a shared DVV representation. Under VV, each sibling stores a complete copy of the causal context, causing metadata to scale approximately with the product of ancestor count and sibling count. DVV stores the shared context once and appends only a single dot per sibling, allowing metadata growth to remain nearly linear in the number of siblings.

At 128 ancestor actors, VV metadata increases extremely rapidly as the number of concurrent siblings grows because the full causal context must be copied into every sibling. DVV avoids this duplication by storing the shared context once and attaching only a small dot per sibling. As concurrency increases, the gap becomes dramatic: VV grows explosively while DVV remains comparatively compact.

Figure 2 summarizes the relative metadata reduction of DVV across the full ancestor-count and sibling-count sweep. For very small contexts, DVV may slightly exceed VV in cost because each sibling requires an explicit dot even when the shared context is tiny. However, once either the ancestor count exceeds roughly 16 actors or the sibling count exceeds four concurrent versions, DVV consistently reduces metadata overhead by large margins. In the largest configurations evaluated, DVV reduced sibling metadata by more than 95% relative to repeated VV storage.

These results show that DVV dramatically reduces redundant metadata across concurrent siblings. However, it does not stop the causal context itself from growing over time as more actors join the system. Experiment 2 therefore studies how VV and DVV behave under long-term membership churn.

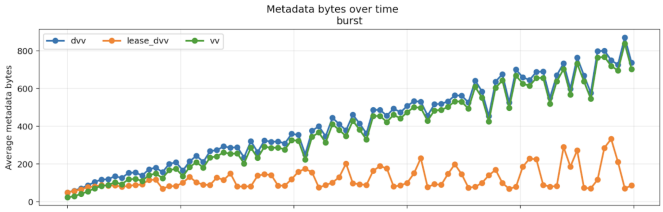

Fig. 3. Experiment 2: Average metadata bytes per write over simulation time under burst churn. DVV and VV grow monotonically as rejoining nodes expand the active context, while Lease-DVV (L=8) remains bounded by continuously pruning expired actor entries at the cost of causal fidelity shown in Figure 4.

B. Experiment 2: VV, DVV, and Lease-DVV Under Churn

Experiment 2 evaluates causal tracking under dynamic membership churn. Figure 3 shows that both VV and DVV experience steady metadata growth under burst churn as replicas repeatedly leave and rejoin the cluster. Rejoining actors expand the active causal context, causing clock metadata to grow throughout the run. DVV remains slightly larger because it additionally stores explicit dots for concurrent writes.

Lease-DVV changes this behavior substantially. By expiring actor entries after a fixed lease duration (L=8), metadata remains bounded throughout the simulation. Under burst churn, Lease-DVV stabilizes near 120 bytes per write, while DVV exceeds 400 bytes. This shows that lease-based pruning is effective at controlling long-term metadata growth.

However, Figure 4 shows that this reduction comes at the cost of causal history. VV and DVV maintain perfect recall throughout the experiment, while Lease-DVV rapidly loses ancestry recall as actor entries expire. Under burst churn, recall quickly drops and later oscillates between roughly 0.1 and 0.6 as the system repeatedly prunes and rebuilds partial ancestry information. In other words, bounded metadata requires forgetting some history.

Figure 5 summarizes this tradeoff across all churn profiles. As churn increases, Lease-DVV achieves larger metadata reductions but also loses more ancestry information. The

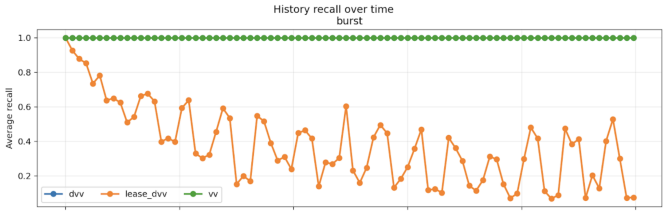

Fig. 4. Experiment 2: Average ancestry recall over simulation time under burst churn. DVV and VV maintain perfect recall of 1.0 throughout. Lease-DVV (L=8) degrades rapidly in the first 200 simulation time units and stabilizes in a noisy band between 0.1 and 0.6, reflecting the oscillating prune-and-rebuild cycle driven by periodic replica departures and rejoins.

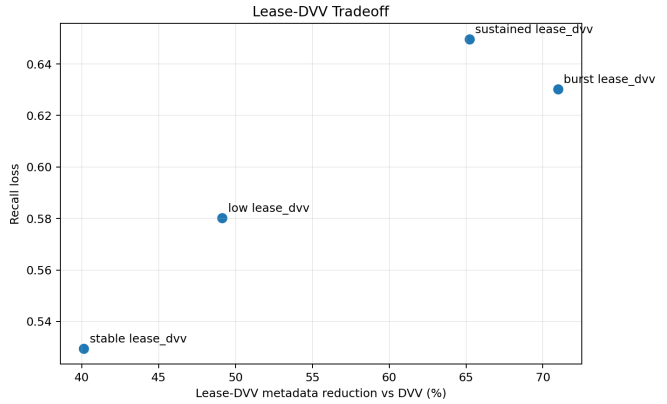

Fig. 5. Experiment 2: Metadata reduction versus recall loss for Lease-DVV (L=8) across all four churn profiles. Each point represents one churn profile. The ideal operating point is the lower-left corner: high metadata reduction with low recall loss. All profiles cluster away from this ideal, with sustained churn producing the worst recall loss despite moderate metadata reduction, and burst churn achieving the highest reduction at the cost of significant ancestry loss.

sustained churn profile produced the worst balance: metadata fell by roughly 65%, but recall dropped below 0.35 because continuous churn prevented actor histories from stabilizing. Burst churn achieved the largest metadata reduction, over 70%, but still suffered substantial recall degradation due to repeated departures and rejoins.

Table III summarizes the aggregate results. VV and DVV preserve perfect recall under every churn profile despite steadily increasing metadata. Lease-DVV consistently achieves the smallest metadata footprint, reducing average per-write metadata by roughly 40–70%, but mean recall falls to 0.35–0.47 across workloads.

Overall, the results show two distinct properties of DVV-based causal tracking. DVVs efficiently represent concurrent siblings by avoiding redundant causal context duplication, but lease-based pruning introduces a strong tradeoff between bounded metadata and preserving causal history.

VII. Conclusion

We evaluated causal metadata mechanisms for replicated key-value stores under dynamic membership churn. Our results show that DVVs preserve the full causal history of exact version vectors while reducing sibling-set metadata overhead

TABLE III
PER-WRITE METADATA AND ANCESTRY RECALL BY CHURN PROFILE AND CLOCK VARIANT. LEASE-DVV USES L=8. **BOLD** INDICATES BEST VALUE PER PROFILE (LOWEST BYTES, HIGHEST RECALL).

		PW bytes		Recall	
Profile	**Clock**	**Mean**	**P95**	**Mean**	**P95**
Stable	VV	102.6	103.0	**1.000**	**1.000**
	DVV	137.0	137.4	**1.000**	**1.000**
	Lease-DVV	**82.0**	**82.7**	0.471	0.480
Low	VV	125.6	134.0	**1.000**	**1.000**
	DVV	159.7	168.0	**1.000**	**1.000**
	Lease-DVV	**81.2**	**84.1**	0.420	0.495
Sustained	VV	210.5	229.6	**1.000**	**1.000**
	DVV	243.8	262.6	**1.000**	**1.000**
	Lease-DVV	**84.8**	**91.1**	0.350	0.387
Burst	VV	384.0	395.8	**1.000**	**1.000**
	DVV	416.1	427.7	**1.000**	**1.000**
	Lease-DVV	**120.8**	**135.8**	0.370	0.394

by over 95% in highly concurrent workloads. However, both VV and DVV exhibit unbounded metadata growth under churn as member sets expand over time.

Lease-DVV successfully bounded metadata growth (121 bytes under burst churn versus 416 bytes for DVV) but introduced substantial ancestry loss, reducing recall from 1.0 to roughly 0.35–0.47 depending on churn profile. An adaptive lease improved recall slightly but did not significantly change the overall metadata-versus-causality tradeoff.

These results suggest that while DVVs efficiently represent concurrency, expiration-based pruning remains a difficult compromise between bounded metadata and preserving causal history. Future work could explore membership-aware pruning that targets only confirmed departures, or hybrid approaches combining lease expiration with periodic anti-entropy synchronization to recover lost ancestry.

References

[1] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.

[2] C. J. Fidge, "Timestamps in message-passing systems that preserve the partial ordering," in *Proc. 11th Australian Computer Science Conference (ACSC)*, 1988, pp. 56–66.

[3] F. Mattern, "Virtual time and global states of distributed systems," in *Parallel and Distributed Algorithms*, M. Cosnard *et al.*, Eds. Amsterdam, The Netherlands: North-Holland, 1989, pp. 215–226.

[4] C. Landes, "Dynamic vector clocks for consistent ordering of events in dynamic distributed applications," in *Proc. IFIP/IEEE Int. Conf. on Distributed Platforms*, 1996, pp. 31–43.

[5] R. S. Torres-Rojas and M. Ahamad, "Plausible clocks: constant size logical clocks for distributed systems," *Distributed Computing*, vol. 12, no. 4, pp. 179–195, Oct. 1999.

[6] P. S. Almeida, C. Baquero, and V. Fonte, "Interval tree clocks: A logical clock for dynamic systems," in *Proc. 12th Int. Conf. Principles of Distributed Systems (OPODIS)*, Luxor, Egypt, 2008, pp. 259–274.

[7] D. S. Parker *et al.*, "Detection of mutual inconsistency in distributed systems," *IEEE Transactions on Software Engineering*, vol. SE-9, no. 3, pp. 240–247, May 1983.

[8] G. DeCandia *et al.*, "Dynamo: amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. on Operating Systems Principles (SOSP)*, Stevenson, WA, USA, 2007, pp. 205–220.

[9] N. Preguiça, C. Baquero, P. S. Almeida, V. Fonte, and R. Gonçalves, "Dotted version vectors: Logical clocks for optimistic replication," in *Proc. Int. Conf. on Networked Systems (NETYS)*, Marrakech, Morocco, 2010, pp. 1–15.

[10] A. Lakshman and P. Malik, "Cassandra: a decentralized structured storage system," *Operating Systems Review*, vol. 44, no. 2, pp. 35–40, Apr. 2010.

[11] Basho Technologies, "Active Anti-Entropy," Riak KV Documentation. Accessed: May 4, 2026. [Online]. Available: https://docs.riak.com/riak/kv/latest/learn/concepts/active-anti-entropy/index.html

APPENDIX

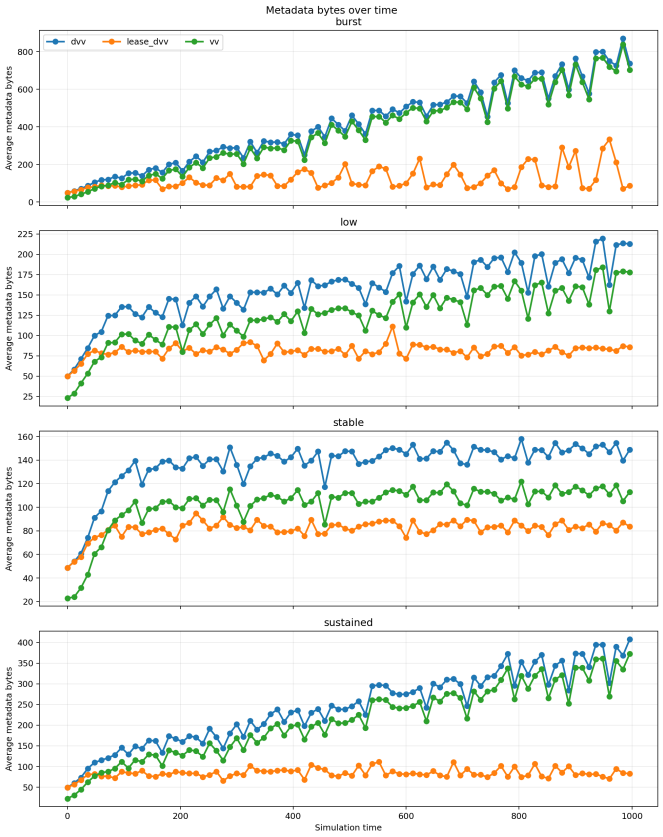

Fig. 6. Full comparison of all churn environments for the plots shown in Figure 3.

The "peak" churn profile models an environment where churn gradually increases, peaks near the middle of the simulation, and then declines. This profile was designed to evaluate whether adaptive leasing could better respond to changing churn conditions over time. As shown in Table IV, DVV preserves perfect causal recall but incurs the highest metadata overhead. Fixed Lease DVV achieves the lowest metadata footprint, but at the cost of substantially reduced recall. Adaptive Lease DVV attempts to dynamically tune lease durations per node using a heuristic based on node silence time, sibling count, observed latency, churn level, and metadata size (see code for detailed equation). While adaptive leasing improves recall relative to the fixed lease configuration for a small metadata increase, our evaluation across multiple lease durations showed that it did not meaningfully improve the overall metadata vs recall trade-off, even though this churn profile was specifically designed to favor this model.

TABLE IV
CLOCK COMPARISON UNDER THE PEAK CHURN PROFILE.

Clock	Mean	P95	Recall	R-P95
Adaptive Lease	114.3	117.9	0.751	0.796
DVV	169.3	178.1	**1.000**	**1.000**
Fixed Lease	**104.7**	**111.8**	0.697	0.739

Comparison of metadata overhead and causal recall with a fixed Lease DVV vs our attempt at an Attaptive Lease DVV.

Fig. 7. Full comparison of all churn environments shown in Figure 4.